

```
$ psql -c "SELECT ... FROM students JOIN courses ON ..."
```

SQL JOIN

// 把兩張表橫向拼起來的五種姿勢

INNER

交集

LEFT

保留左表

RIGHT

保留右表

FULL

聯集

CROSS

笛卡爾積

\$ cat query-plan.md

今日路線

01 / 五種 JOIN

交集、保左、保右、聯集、全組合

02 / 決策

比較表與決策樹

03 / 經典陷阱

ON vs WHERE 的差異

04 / 實戰

多對多、效能、最佳實踐

快速判斷法：「右表沒匹配時，左表的資料還要顯示嗎？」要 = LEFT，不要 = INNER。

PART 01

五種 JOIN

學生表 ID: 1,2,3,4 × 選課表 ID: 2,3,5

INNER JOIN : 找交集

```
1 SELECT
2     s.student_id,
3     s.name AS student_name,
4     c.course_name
5 FROM students AS s
6 INNER JOIN courses AS c
7     ON s.student_id = c.student_id;
```

比喻

「既註冊又選課」的學生才出現——像數學的交集

RESULT SET

學生表 ID : 1, 2, 3, 4

選課表 ID : 2, 3, 5

→ 結果只剩 ID : 2, 3

x 沒選課的學生 (1, 4) 不見了

x 沒人選的課程 (5) 也不見了

LEFT JOIN：保留左表全部

```
1  SELECT
2      s.name,
3      c.course_name,
4      CASE
5          WHEN c.course_name IS NULL
6          THEN '未選課' ELSE '已選課'
7      END AS status
8  FROM students AS s
9  LEFT JOIN courses AS c
10     ON s.student_id = c.student_id;
```

RESULT SET

- 結果：ID 1, 2, 3, 4 全部保留
- 1 和 4 的課程欄位是 NULL

比喻

老師統計「所有學生」的選課狀況——沒選課的也要列在名單上

LEFT JOIN 的兩大絕活

絕活 1：找出「沒有」的資料

```
1 SELECT c.customer_name
2 FROM customers c
3 LEFT JOIN orders o
4     ON c.customer_id = o.customer_id
5 WHERE o.order_id IS NULL;
6 -- 沒有訂單的客戶
```

絕活 2：統計關聯數量

```
1 SELECT s.name,
2        COUNT(c.course_id) AS course_count
3 FROM students s
4 LEFT JOIN courses c
5     ON s.student_id = c.student_id
6 GROUP BY s.student_id, s.name;
7 -- 沒選課的學生 count = 0
```

LEFT JOIN + IS NULL 是資料完整性檢查的標準招式—INNER JOIN 做不到。

RIGHT、FULL OUTER、CROSS

RIGHT JOIN

保留右表全部。實務少用：

`A LEFT JOIN B ≡ B RIGHT JOIN A`，統一用 LEFT 更好讀

FULL OUTER JOIN

兩邊全保留，沒匹配補 NULL。

MySQL 不支援：用 LEFT ∪ RIGHT (UNION) 模擬

CROSS JOIN

笛卡爾積：3 筆 × 4 筆 = 12 筆。

合法用途：產生「尺寸 × 顏色」所有組合

```
1  -- MySQL 模擬 FULL OUTER JOIN
2  SELECT * FROM students s LEFT JOIN courses c ON s.student_id = c.student_id
3  UNION
4  SELECT * FROM students s RIGHT JOIN courses c ON s.student_id = c.student_id;
```

PART 02

怎麼選？

一張表 + 一棵決策樹

五種 JOIN 一覽

JOIN 類型	左表	右表	無匹配時	使用頻率
INNER JOIN	有匹配的	有匹配的	不顯示	★★★★★
LEFT JOIN	全部	有匹配的	右表補 NULL	★★★★★
RIGHT JOIN	有匹配的	全部	左表補 NULL	★★
FULL OUTER	全部	全部	另一邊補 NULL	★★
CROSS JOIN	全部	全部	產生所有組合	★

\$ 90% 的查詢只用得到 INNER 和 LEFT — 先把這兩個練熟。

決策樹



PART 03

經典陷阱：ON vs WHERE

90% 的 LEFT JOIN bug 都在這

CHECKPOINT_01

LEFT JOIN courses c ... WHERE c.course_name = 'Math' 的結果是？

- A 所有學生都會列出，沒修 Math 的顯示 NULL
- B 只剩有修 Math 的學生——LEFT JOIN 被 WHERE 變成 INNER 效果
- C 語法錯誤：LEFT JOIN 之後不能接 WHERE

條件放哪裡，結果差很大

```
1  -- ❌ WHERE : JOIN 完才過濾 → NULL 列被濾掉，變 INNER 效果
2  SELECT * FROM students s
3  LEFT JOIN courses c ON s.student_id = c.student_id
4  WHERE c.course_name = 'Math';    -- 只剩有修 Math 的學生
5
6  -- ✅ ON : JOIN 階段就過濾 → 所有學生保留
7  SELECT * FROM students s
8  LEFT JOIN courses c
9      ON s.student_id = c.student_id
10     AND c.course_name = 'Math'; -- 沒修 Math 的顯示 NULL
11
12 -- ✅ 要找「沒修課的學生」→ IS NULL 放 WHERE
13 SELECT * FROM students s
14 LEFT JOIN courses c ON s.student_id = c.student_id
15 WHERE c.student_id IS NULL;
```

ON = JOIN 階段過濾；WHERE = JOIN 完成後過濾。

PART 04

實戰

多對多、效能、最佳實踐

多對多：透過中介表

```
1  -- 學生選課系統：多對多關係
2  -- students ← enrollments → courses
3
4  SELECT
5      s.name AS student_name,
6      c.course_name,
7      e.enrollment_date
8  FROM students s
9  INNER JOIN enrollments e ON s.student_id = e.student_id
10 INNER JOIN courses c ON e.course_id = c.course_id;
```

中介表設計

UNIQUE KEY (student_id, course_id) 防重複選課

串聯 JOIN

兩段 INNER JOIN 走過中介表，就是多對多查詢

為什麼我的 JOIN 很慢？

兇手 1：忘記寫 ON

```
1  -- ❌ 笛卡爾積
2  SELECT * FROM students s
3  INNER JOIN courses c;
4  -- 結果數 = 學生數 × 課程數
```

兇手 2：關聯欄位沒索引

```
1  -- ✅ 為關聯欄位建索引
2  CREATE INDEX idx_courses_student_id
3  ON courses(student_id);
```

DEBUG FLOW

1. EXPLAIN 看執行計劃 → 2. 檢查 ON 條件 → 3. 檢查關聯欄位索引 → 4. JOIN 超過 5 張表就重新想設計

最佳實踐

DO

- ✓ 用表格別名：`students AS s`
- ✓ 明確指定欄位，不用 `SELECT *`
- ✓ 統一用 `LEFT JOIN` 取代 `RIGHT JOIN`

DON'T

- × 忘記 `ON` 條件 → 笛卡爾積爆炸
- × 該 `LEFT` 用 `INNER` → 資料默默消失
- × 過濾條件誤放 `WHERE` → `LEFT` 變 `INNER`

今日重點

INNER = 交集

只留兩邊都匹配的

LEFT = 保左

最常用；+ IS NULL 找缺失資料

ON vs WHERE

JOIN 階段過濾 vs JOIN 後過濾

效能

關聯欄位建索引；別忘 ON 條件

「右表沒匹配時還要顯示左表嗎？」—— 要 = LEFT, 不要 = INNER。

Tables joined.

下一步：資料庫索引，讓 JOIN 快起來

```
$ EXPLAIN SELECT ... FROM a LEFT JOIN b ON ...  
inner → left → on/where → index
```